
CS3460: Data Structures

Lab 04: Binary Search Trees

Total Points: 100

Problems

Building a Binary Search Tree (60 points) **2**

Runway Reservation (40 points) **4**

Notes on Grading: Grading will be handled via Gradescope. Any file you submit must begin with the following package statement to be accepted:

```
1 package student;
```

Your code will be compiled and run against a series of tests. You may submit as many times as you would like before the assignment closes. To earn full credit, you must submit to both Gradescope and AsULearn before the assignment deadline.

Important Note: You are not allowed to use any classes or code from the Java Collections library (built-in Java data structures). The purpose of these assignments is to build these data structures from first principles. Programs which use any of these libraries will receive zero points. This only applies to Collections, and does not include classes like **Scanner** or other utility classes.

Submission: Please submit the files `BST.java` and `RunwayReservation.java`.

1. **Building a Binary Search Tree (60 points):** A *binary search tree* is a node-based data structure that encodes a sorted ordering on its inserted keys. It is capable of answering queries beyond that of what hash tables are capable of.

For this assignment we will be building a binary search tree keyed on an integer `time`. You have been provided a `Node` class with a data field `flight`, which contains a time and airline information. Do not modify the `Node.java` or `Flight.java` files.

You will write a file `BST.java` that implements a binary search tree using the provided `Node` class. You will need to implement the following methods:

- (a) `public void insert(Flight flight)`: insert a `Flight` into the binary search tree, keyed on the `Flight`'s `time` field.
- (b) `public Node pred(int time)`: return the `Node` within the binary search tree that is the predecessor of the value `time` (i.e., the largest integer smaller than `time`). Return `null` if there is no predecessor. This can be implemented iteratively.
- (c) `public Node succ(int time)`: return the `Node` within the binary search tree that is the successor of the value `time` (i.e., the smallest integer larger than `time`). Return `null` if there is no successor. This can be implemented iteratively.
- (d) `public Node min()`: return the `Node` with the smallest key in the tree. Return `null` if the tree is empty. This can be implemented iteratively.
- (e) `public Node max()`: return the `Node` with the largest key in the tree. Return `null` if the tree is empty. This can be implemented iteratively.
- (f) `public void delete(int time)`: remove the `Node` with the key `time` (if such a node is present, otherwise do nothing).

For many of these methods, they may be best implemented as a public wrapper that calls a private, recursive implementation of the method. The public interface for the class will not have references to internal nodes generally, so any method that takes a `Node` as an argument will likely be private.

`delete(time)` can be implemented recursively. Suppose we try to delete from a subtree rooted at x . The recursive method will return the root of the subtree resulting from calling `delete(node, time)`. If `x == null`, we return `null`, otherwise:

- (a) x is a leaf: Then x has no children. Therefore, we return `null`. This gets updated by the parent node to effectively remove the node from the tree.

- (b) **x has one child y :** Then return y . This will update the parent's child pointer to y to remove x from the tree.
- (c) **x has two children:** Find $s = \text{succ}(x)$, then copy the data fields from s into x , and delete the value of s from x 's right subtree.

Two recursive cases remain, to walk down the subtree to the left or right to find the node in question, based on a comparison of the node's key and the value being searched for. You may want to build your own driver class to test your BST methods.

2. **Runway Reservation (40 points):** Now we will implement a single-runway reservation system. Airlines make request reservations for the runway at specific times. In order to accommodate these requests, and to account for delays and buffer, we allow a grace period of length k between successive uses of the runway. All requests are fulfilled first-come, first-served. You will write a program to output all successful requests for runway allocation. That is to say, some requests will be unsuccessful.

The provided file `Flight.java` contains a class representing a single flight. Please make sure you understand how this class works.

The provided file `RunwayReservation.java` contains a `main()` method that reads through an input file (through `System.in`) and loads the events into an array of `Request` objects. The input file contains two numbers, n and k on the first line, where n represents the total number of events and k represents the grace period allowed between reservations. What follows are n lines containing the events described above. These events are either reservation requests (denoted by the “r” command) or an advance time event (denoted by the “t” command). Each “r” command includes information about the time requested and the airline information for the request. Each “t” command includes some integer amount to advance the *current time* by. See the sample input listed later for better clarity.

The provided `main()` automatically passes these requests methods that you will need to implement. Use a binary search tree keyed on times to store all valid, successful requests. Requests are always processed in order.

The methods you will need to implement are:

- `public RunwayReservation(int grace)` - this is the constructor for your class. The “grace period” length will be passed in as an argument. This is the buffer.
- `public boolean requestReservation(Flight flight)` - for r-type events, this method will be called. Check if the request is valid with respect to already allocated reservations (taking into account the length- k grace period). If the request is valid, add it to the BST. Return true or false indicating whether the reservation was made.
- `public List<Flight> advanceTime(int t)` - for t-type events, this method will be called. Update the current time, and **remove** all requests from the BST with times up to and including the current time. Add these removed flights to a `List<Flight>` and return it. For this portion, you may use the List classes from Java Collections. These are successful requests who used the runway already.
- `public int getCurrentTime()` - return the current time. Time starts at 0 and updates with every t-type event.

For each reservation request for a time of t , we can check if it is valid in $O(\lg n)$ time by checking `pred(t)` and `succ(t)` to determine if there is enough buffer. Keep in mind, reservations that have already passed due to a `t`-type event are still afforded a length- k buffer. Throughout the entirety of the algorithm, each request is inserted at most one time, and removed at most once, which takes $O(\lg n)$ each, for a total running time of $O(n \lg n)$.

`small-flight.in`

```
12 5
r 20 UA 1545 CLT IAH
r 9 UA 1714 CLT IAH
r 6 AA 1141 CLT MIA
r 5 B6 725 CLT BQN
r 10 DL 461 CLT ATL
r 6 B6 79 CLT MCO
t 7
r 25 UA 1696 CLT ORD
r 7 B6 507 CLT FLL
t 10
r 24 EV 5708 CLT IAD
t 8
```

`small-flight.out`

```
Current time = 7 units
Current time = 17 units
UA 1714 CLT IAH
Current time = 25 units
UA 1545 CLT IAH
UA 1696 CLT ORD
```